



Salicornia Digital

Documentación Técnica y Funcional

PRACTICAS-SALICORNIA-DIGITAL · Marismas Biomed

Preparado para responsables y futuros programadores

8 de junio de 2026

1. Resumen ejecutivo

Salicornia Digital es una aplicación web para gestionar el cultivo de salicornia en esteros. Cubre la gestión de infraestructura, planchas, macetas, cuaderno de campo, incidencias, IoT, cosechas, trazabilidad, datos ambientales y usuarios.

La aplicación combina:

- Frontend JavaScript estático con estado local en el navegador.
- Backend Express/MySQL centrado en autenticación, sesiones, recuperación de contraseña y gestión de usuarios.

Novedad: el módulo Infraestructura incluye ahora un formulario «Nueva plancha» que crea y guarda planchas en el estado local. La nueva plancha se selecciona automáticamente, aparece en el mapa visual y en el desplegable global, y queda persistida en localStorage.

2. Estructura del proyecto

server.js	Servidor Express, conexión MySQL, sesiones, rutas de autenticación y usuarios.
public/	Frontend servido por Express: index.html, app.js, styles.css, PWA manifest, service worker, users.js e iconos.
salicornia-digital/	Copia estática del frontend con servidor-local.js. Sincronizada con public/app.js.
Base_Datos-Marismas.sql	Esquema e inserts iniciales de MySQL/MariaDB.
seed.js	Crea o actualiza usuarios demo con bcrypt, jerarquía y módulos.
.env.example	Plantilla de configuración local.
package.json	Scripts npm y dependencias del backend.

3. Puesta en marcha

Sigue estos pasos en orden para levantar el proyecto en un entorno nuevo:

- Instalar MySQL o MariaDB e importar el esquema: *Base_Datos-Marismas.sql*
- Copiar .env.example a .env y ajustar las variables (ver sección 4).
- Instalar dependencias:

```
npm install
```

- Generar usuarios demo:

```
npm run seed
```

- Arrancar el servidor:

```
npm start          # producción
```

```
npm run dev          # desarrollo con reinicio automático (nodemon)
```

La aplicación quedará disponible en <http://localhost:3000> por defecto.

4. Variables de entorno

Base de datos

DB_HOST	Host de MySQL / MariaDB.
DB_USER	Usuario de base de datos.
DB_PASSWORD	Contraseña de base de datos.
DB_NAME	Nombre de la base de datos.
DB_POOL_SIZE	Tamaño del pool de conexiones (por defecto: 10).

Sesión y servidor

SESSION_SECRET	Obligatorio en producción: cadena larga, aleatoria y privada.
SESSION_MAX_AGE_MS	Caducidad de la cookie de sesión (por defecto: 15 minutos).
PORT	Puerto HTTP (por defecto: 3000).
APP_URL	URL base usada en los enlaces de recuperación de contraseña.
NODE_ENV	production activa cookies secure en las sesiones.

Correo electrónico (SMTP)

SMTP_HOST	Host del servidor SMTP. Si está vacío, los códigos se muestran por consola.
SMTP_PORT	Puerto SMTP.
SMTP_SECURE	true para TLS.
SMTP_USER	Usuario SMTP.
SMTP_PASS	Contraseña SMTP.
MAIL_FROM	Dirección de remitente.

5. Backend y API

El backend usa Express, cors, express-session, bcrypt, mysql2/promise, dotenv, crypto y nodemailer. Sirve `public/` como estático y redirige cualquier ruta desconocida a `public/index.html`.

Endpoints de autenticación

GET /api/health	Comprueba la conexión con la base de datos.
GET /api/auth/me	Devuelve el usuario de sesión activo o null.
POST /api/auth/login	Valida usuario/email y contraseña (bcrypt).
POST /api/auth/register	Crea un usuario básico con módulo trace.
POST /api/auth/forgot-password	Genera código de 6 dígitos y token hash con caducidad de 10 minutos.
POST /api/auth/reset-password	Valida código/token y cambia la contraseña.
POST /api/auth/logout	Destruye la sesión y limpia la cookie.

Endpoints de usuarios

GET /api/users	Lista usuarios gestionables por admin/superadmin.
POST /api/users	Crea usuarios con rol, jerarquía y módulos.
PATCH /api/users/:username	Actualiza email, rol, jerarquía, módulos y estado activo.
DELETE /api/users/:username	Elimina usuarios gestionables.

6. Frontend

El frontend está implementado en public/app.js sin framework: renderiza HTML con template strings, escucha eventos globales y guarda el estado de cultivo en localStorage bajo la clave salicornia-digital-state-v1.

Vistas principales

- Inicio
- Infraestructura
- Datos ambientales
- Cuaderno de campo
- Incidencias
- IoT
- Cosechas
- Trazabilidad
- Gestionar usuarios

Comportamientos clave

- **Autenticación** *Intenta usar el backend si no se abre como file://. Si el backend no responde, cae a usuarios demo del frontend.*
- **Inactividad** *La sesión cliente caduca tras 15 minutos sin eventos de actividad.*
- **Selección de plancha** *Se mantiene en selectedPlancha, se refleja en el hash de URL y alimenta formularios, QR, macetas, gráficas y filtros.*

- Códigos QR *Se generan identificadores QR para plancha, maceta y registros de campo.*
- Datos ambientales *Sincronización automática con Open-Meteo Forecast + Marine API en el cliente.*

7. Módulo de planchas

El formulario «Nueva plancha» se ubica en Infraestructura, el módulo que ya muestra secciones, planchas, macetas y el detalle técnico. El formulario aparece debajo del mapa y detalle.

Campos del formulario

- ID plancha — sugerido automáticamente como siguiente PL-XXX disponible.
- Sección
- Superficie en m²
- Número de macetas
- Estado

Validación y guardado

- Acepta IDs con formato PL- seguido de al menos tres dígitos.
- Bloquea duplicados comprobando state.planchas.
- Inserta en state.planchas y llama saveState(), persistiendo en localStorage.
- Post-guardado: selecciona la nueva plancha, actualiza el hash de URL, re-renderiza la app y muestra confirmación.

 Limitación actual: los datos no se escriben en la tabla SQL planchas. Para persistencia multiusuario real habría que añadir endpoints backend y conectar el frontend a MySQL.

8. Base de datos

El SQL define un modelo completo, aunque el backend actual solo usa directamente las tablas usuarios y password_reset_tokens.

Tablas principales

instalaciones	Ubicaciones o centros de cultivo.
secciones	Áreas dentro de una instalación (código tipo S-001).
planchas	Unidades de cultivo asociadas a sección, con identificador único.
macetas	Posiciones dentro de una plancha.
registros_campo	Mediciones de crecimiento, biomasa y notas.
fotos	Imágenes e incidencias asociadas a plancha/maceta.
sensores_lecturas	Lecturas IoT por plancha.
datos_ambientales	Mareas, viento, lluvia, temperatura y parámetros

	ambientales por fecha.
cosechas	Lotes cosechados: canal, marea y peso.
lotes_trazabilidad	Lavado, secado, envasado, humedad, destino y estado.
usuarios	Credenciales, rol, jerarquía, módulos_json, activo y último acceso.
password_reset_tokens	Tokens de recuperación segura de contraseñas (gestionados por server.js).

9. Usuarios, roles y permisos

La jerarquía técnica es: user < admin < superadmin. Superadmin tiene acceso completo y puede gestionar admins y usuarios. Admin puede gestionar usuarios de menor jerarquía según las reglas de canManageUser.

Usuario	Rol	Módulos asignados
admin	superadmin	Todos los módulos
david	admin	dashboard, trace
alejandro	user	ambient, trace
gonzalo	user	iot, trace
javi	user	infra, trace
jesus	user	fieldbook, trace
guille	user	incidents, trace
anibal	user	harvests, trace

Todos los usuarios demo se crean con la contraseña salicornia123 mediante npm run seed. En producción deben cambiarse las contraseñas y revisarse los permisos.

10. Seguridad

- Contraseñas almacenadas con bcrypt cost 12.
- Cookies httpOnly, sameSite lax y secure cuando NODE_ENV=production.
- Recuperación con código y token guardados como SHA-256, caducidad de 10 minutos y marcado usado_en.
- CORS permite origin configurable; revisar CORS_ORIGIN en producción.
- El secreto de sesión por defecto no debe usarse fuera de desarrollo.
- No guardar secretos reales en el repositorio. Mantener .env fuera de Git.
- La gestión de usuarios protege operaciones con requireAuth, requireUserManager y canManageUser.

11. PWA y recursos estáticos

El proyecto incluye manifest.webmanifest, sw.js e iconos. El frontend puede funcionar como aplicación instalable.

⚠ Revisar el caché del service worker al desplegar una nueva versión para evitar que los usuarios vean JavaScript antiguo.

12. Mantenimiento y deuda técnica

- **Duplicidad public/ y salicornia-digital/** *Hay dos copias del frontend. Conviene elegir una única fuente o automatizar la sincronización.*
- **Persistencia de cultivo** *Planchas, cuaderno, fotos, sensores, cosechas y trazabilidad viven en localStorage. Esto no sincroniza entre dispositivos ni usuarios.*
- **Endpoints CRUD pendientes** *La base SQL ya permite persistencia real. El siguiente paso natural es crear endpoints para planchas, registros, fotos, sensores, cosechas y trazabilidad.*
- **CSS grande con imágenes embebidas** *Para mantenimiento, valorar separar assets y organizar CSS por módulos.*
- **Template strings extensos** *Si el frontend crece, considerar componentes o un framework ligero.*
- **Pruebas automáticas** *Al menos: node --check, pruebas de rutas auth y pruebas end-to-end básicas de login / crear plancha.*
- **Backup y migraciones** *Revisar estrategia de backup de MySQL y migraciones versionadas antes de usar en producción.*

13. Comandos útiles

```
npm install           # Instalar dependencias
npm run seed          # Crear / actualizar usuarios demo
npm start             # Arrancar servidor Express
npm run dev           # Arrancar con nodemon (desarrollo)
node --check public/app.js  # Validar sintaxis del frontend
node --check server.js    # Validar sintaxis del backend
```

14. Archivos modificados en esta entrega

Los cambios de esta entrega afectan únicamente al módulo de planchas. No se han modificado server.js ni el esquema SQL, ya que la arquitectura actual guarda los datos de cultivo en localStorage.

public/app.js	Helpers nextPlanchaId y sectionOptions, formulario «Nueva plancha» en renderInfra y manejador submit plancha-form.
salicornia-digital/app.js	Sincronizado con el mismo cambio para la copia estática.